Kernel frameworks for device drivers



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

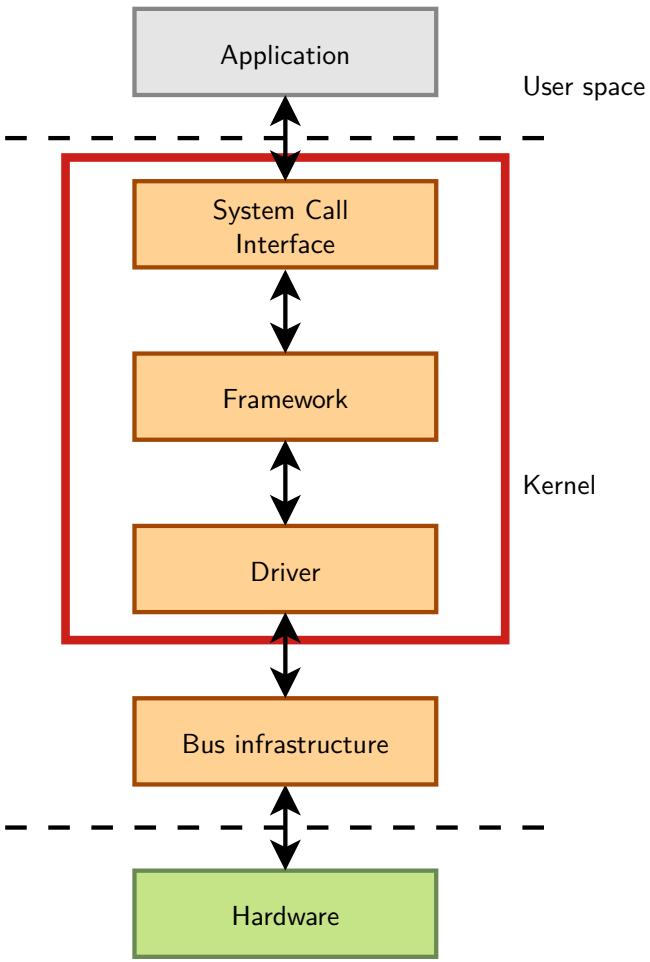


Kernel and Device Drivers

In Linux, a driver is always interfacing with:

- a **framework** that allows the driver to expose the hardware features to user space applications.
- a **bus infrastructure**, part of the device model, to detect/communicate with the hardware.

This section focuses on the *kernel frameworks*, while the *bus infrastructure* was covered earlier in this training.





User space vision of devices



Types of devices

Under Linux, there are essentially four types of devices:

- **Network devices**. They are represented as network interfaces, visible in user space using `ip a`
- **Block devices**. They are used to provide user space applications access to raw storage devices (hard disks, USB keys). They are visible to the applications as *device files* in `/dev`.
- **Character devices**. They are used to provide user space applications access to all other types of devices (input, sound, graphics, serial, etc.). They are also visible to the applications as *device files* in `/dev`.
- **Sysfs devices**. They don't have any of the above user space interfaces, only a representation in sysfs. "Internal" device drivers fall under this (e.g. pinctrl), but also some user-space accessible devices. E.g. gpio (deprecated), IIO (Industrial I/O).

→ Most devices are *character devices*, so we will study these in more details.



Devices: everything is a file

- A very important UNIX design decision was to represent most *system objects* as files
- It allows applications to manipulate all *system objects* with the normal file API (`open`, `read`, `write`, `close`, etc.)
- So, devices had to be represented as files to the applications
- This is done through a special artifact called a **device file**
- It is a special type of file, associating a file name visible to user space applications to a triplet the kernel understands:
  - *type*: `char` or `block`
  - *major*: class of the device
  - *minor*: unique identifier in a class
- All *device files* are by convention stored in the `/dev` directory



Device files examples

Example of device files in a Linux system

```
$ ls -l /dev/ttyS0 /dev/tty1 /dev/sda /dev/sda1 /dev/sda2 /dev/sdc1 /dev/zero
brw-rw---- 1 root disk    8,  0 2011-05-27 08:56 /dev/sda
brw-rw---- 1 root disk    8,  1 2011-05-27 08:56 /dev/sda1
brw-rw---- 1 root disk    8,  2 2011-05-27 08:56 /dev/sda2
brw-rw---- 1 root disk    8, 32 2011-05-27 08:56 /dev/sdc
crw------- 1 root root    4,  1 2011-05-27 08:57 /dev/tty1
crw-rw---- 1 root dialout 4, 64 2011-05-27 08:56 /dev/ttyS0
crw-rw-rw- 1 root root    1,  5 2011-05-27 08:56 /dev/zero
```

Example C code that uses the usual file API to write data to a serial port

```c
int fd;
fd = open("/dev/ttyS0", O_RDWR);
write(fd, "Hello", 5);
close(fd);
```



Creating device files

- Before Linux 2.6.32, on basic Linux systems, the device files had to be created manually using the `mknod` command
  - `mknod /dev/<device> [c|b] major minor`
  - Needed root privileges
  - Coherency between device files and devices handled by the kernel was left to the system developer
- The `devtmpfs` virtual filesystem can be mounted on `/dev` and contains all the devices registered to kernel frameworks. The CONFIG_DEVTMPFS_MOUNT kernel configuration option makes the kernel mount it automatically at boot time, except when booting on an initramfs.
- `devtmpfs` can be supplemented by userspace tools like `udev` or `mdev` to adjust permission/ownership, load kernel modules automatically and create symbolic links to devices.



Character drivers

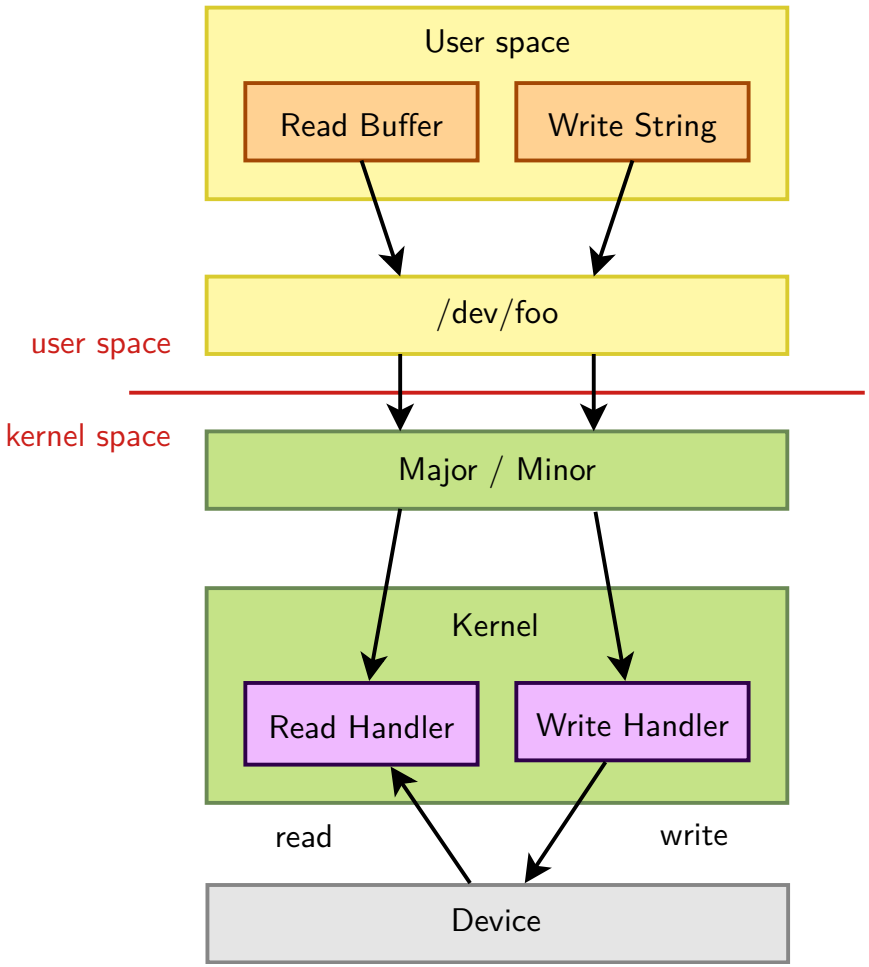


A character driver in the kernel

- From the point of view of an application, a *character device* is essentially a **file**.
- Character device drivers therefore implement **operations** that let applications think the device is a file.
- In order to achieve this, a character driver implements the operations it wants from the struct file_operations structure: `read`, `write`, `ioctl`, etc.
- The Linux filesystem layer will ensure that the driver's operations are called when a user space application makes the corresponding system call.



From user space to the kernel: character devices





File operations

Here are the most important operations for a character driver, from the definition of struct file_operations:

```
struct file_operations {
    struct module *owner;
    ssize_t (*read) (struct file *, char __user *,
        size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *,
        size_t, loff_t *);
    long (*unlocked_ioctl) (struct file *, unsigned int,
        unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*release) (struct inode *, struct file *);
    ...
};
```

Many operations exist, they are all optional.



open() and release()

- `int open(struct inode *i, struct file *f)`
  - Called when user space opens the device file.
  - Only implement this function when you do something special with the device at `open()` time.
  - `struct inode` is a structure that uniquely represents a file in the filesystem (be it a regular file, a directory, a symbolic link, a character or block device)
  - `struct file` is a structure created every time a file is opened. Several file structures can point to the same `inode` structure.
    - Contains information like the current position, the opening mode, etc.
    - Has a `void *private_data` pointer that one can freely use.
    - A pointer to the `file` structure is passed to all other operations
- `int release(struct inode *i, struct file *f)`
  - Called when user space closes the file.
  - Only implement this function when you do something special with the device at `close()` time.



read() and write()

- `ssize_t read(struct file *f, char __user *buf, size_t sz, loff_t *off)`
  - Called when user space uses the `read()` system call on the device.
  - Must read data from the device, write at most `sz` bytes to the user space buffer `buf`, and update the current position in the file `off`. `f` is a pointer to the same file structure that was passed in the `open()` operation
  - Must return the number of bytes read. `0` is usually interpreted by userspace as the end of the file.
  - On UNIX, `read()` operations typically block when there isn't enough data to read from the device
- `ssize_t write(struct file *f, const char __user *buf, size_t sz, loff_t *off)`
  - Called when user space uses the `write()` system call on the device
  - The opposite of `read`, must read at most `sz` bytes from `buf`, write it to the device, update `off` and return the number of bytes written.



Exchanging data with user space 1/3

- Kernel code isn't allowed to directly access user space memory, using memcpy() or direct pointer dereferencing
  - User pointer dereferencing is disabled by default to make it harder to exploit vulnerabilities.
  - If the address passed by the application was invalid, the kernel could segfault.
  - **Never** trust user space. A malicious application could pass a kernel address which you could overwrite with device data (`read` case), or which you could dump to the device (`write` case).
  - Doing so does not work on some architectures anyway.
- To keep the kernel code portable, secure, and have proper error handling, your driver must use special kernel functions to exchange data with user space.

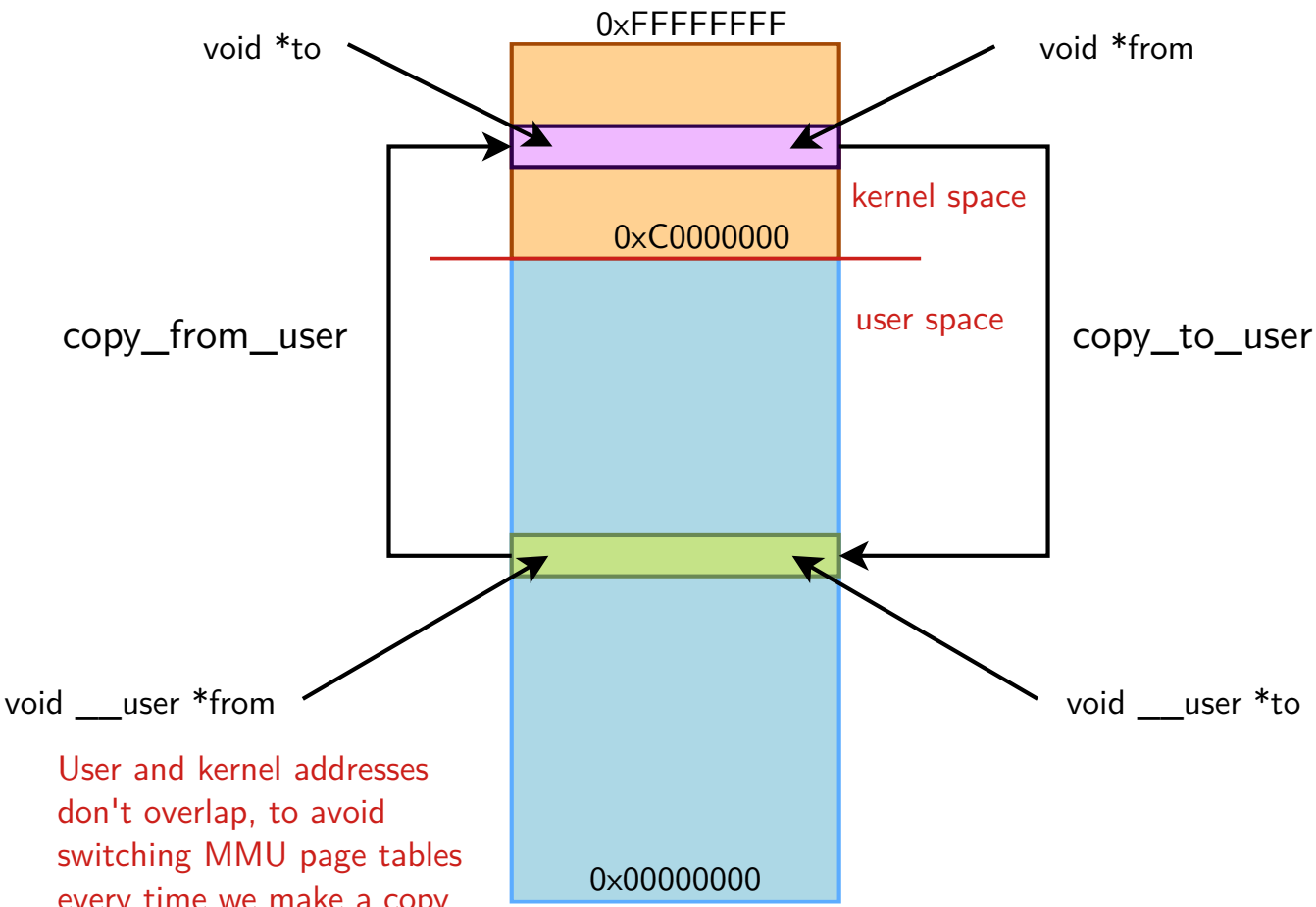


Exchanging data with user space 2/3

- A single value
  - `get_user(v, p);`
    - The kernel variable `v` gets the value pointed by the user space pointer `p`
  - `put_user(v, p);`
    - The value pointed by the user space pointer `p` is set to the contents of the kernel variable `v`.
- A buffer
  - ```
    unsigned long copy_to_user(void __user *to, const void *from,
                               unsigned long n);
    ```
  - ```
    unsigned long copy_from_user(void *to, const void __user *from,
                                 unsigned long n);
    ```
- The return value must be checked. Zero on success, non-zero on failure. If non-zero, the convention is to return -EFAULT.



Exchanging data with user space 3/3





Zero copy access to user memory

- Having to copy data to or from an intermediate kernel buffer can become expensive when the amount of data to transfer is large (video).
- *Zero copy* options are possible:
  - `mmap()` system call to allow user space to directly access memory mapped I/O space. See our `mmap()` chapter.
  - get_user_pages() and related functions to get a mapping to user pages without having to copy them.



unlocked_ioctl()

- `long unlocked_ioctl(struct file *f, unsigned int cmd, unsigned long arg)`
  - Associated to the `ioctl()` system call.
  - Called unlocked because it didn't hold the Big Kernel Lock (gone now).
  - Allows to extend the driver capabilities beyond the limited read/write API.
  - For example: changing the speed of a serial port, setting video output format, querying a device serial number... Used extensively in the V4L2 (video) and ALSA (sound) driver frameworks.
  - `cmd` is a number identifying the operation to perform.
    See driver-api/ioctl for the recommended way of choosing `cmd` numbers.
  - `arg` is the optional argument passed as third argument of the `ioctl()` system call. Can be an integer, an address, etc.
  - The semantic of `cmd` and `arg` is driver-specific.



ioctl() example: kernel side

```
#include <linux/phantom.h>

static long phantom_ioctl(struct file *file, unsigned int cmd,
                          unsigned long arg)
{
        struct phm_reg r;
        void __user *argp = (void __user *)arg;

        switch (cmd) {
        case PHN_SET_REG:
                if (copy_from_user(&r, argp, sizeof(r)))
                        return -EFAULT;
                /* Do something */
                break;
        ...
        case PHN_GET_REG:
                if (copy_to_user(argp, &r, sizeof(r)))
                        return -EFAULT;
                /* Do something */
                break;
        ...
        default:
                return -ENOTTY;
        }

        return 0;
}
```

Selected excerpt from drivers/misc/phantom.c



Ioctl() Example: Application Side

```c
#include <linux/phantom.h>

int main(void)
{
        int fd, ret;
        struct phm_reg reg;

        fd = open("/dev/phantom");
        assert(fd > 0);

        reg.field1 = 42;
        reg.field2 = 67;

        ret = ioctl(fd, PHN_SET_REG, &reg);
        assert(ret == 0);

        return 0;
}
```